

Organizing Documented Processes

Biplav Srivastava Debdoot Mukherjee
 IBM India Research Laboratory
 New Delhi and Bangalore, India
 {sbiplav, debdmukh}@in.ibm.com

Abstract

A frequent hurdle in applying Business Process Management (BPM) to large enterprises is that, since business processes are not only numerous but also documented in an engagement in multiple representations, it is difficult to work with the documented ‘as-is’ or ‘to-be’ state of the business, leave aside make accurate transformation decisions. In this paper, we consider the problem of how to reconcile and organize documented information about processes into groups that convey inter-process similarity. The discovered knowledge can be used for many applications like search, e.g., find all requirements across all available processes that are similar to those of the “Account Receivable” process. The method has been tested on a dataset consisting of hundreds of processes documented in Word and Visio, where from it could find process clusters that significantly boosted search performance.

1 Introduction

Business Process Management (BPM) envisages business processes at the center of all activities including Information Technology (IT) transformations. Organizing documented business processes becomes important because enterprises want to analyze existing documentation to gain insights. However, applying BPM during transformational exercises in large enterprises remains a challenge since dealing with voluminous documentation of the ‘as-is’ or ‘to-be’ state of business has proven to be particularly difficult. The business processes are not only numerous but also documented in multiple representations. For example, a process’ textual description can be in Word while the behavior can also be represented as a flow diagram in Visio with overlapping and additional content. Also, since drafting business process information generally involves large teams, inconsistencies often creep in var-

ious descriptions of the same process. Methods and tools provisioning a consolidated single view of business process related information can give a huge boost to adoption of BPM principles. Moreover, we observe that a host of design information viz. requirements, performance indicators, gap specifications, which are useful in downstream design and realization of IT systems, are captured alongside business process descriptions in process documentation. Thus, linking such ancillary artifacts to organized process information can yield a more comprehensive framework wherein we can also harvest critical non-process artifacts that are effective in context of a business process.

Research in BPM has focused on efficiently harvesting process modeling information captured in BPMN, YAWL and other workflow languages. Techniques to precisely measure similarity in process structure and semantics have evolved over time [2, 4]. Process mining research has addressed the problem of extracting process structure from execution logs [8]. But, discovering and organizing process information present in blueprint documents has not received significant attention. Also, the power of maintaining non-process design artifacts alongside process information has remained understated. In this paper, we propose a framework through which enterprises can gain insight from legacy content of previous projects by consolidating different manifestations of process information, organizing similar business processes into groups and reusing it productively.

Our solution consists of first reconciling process description from multiple representations to produce an enhanced process definition that corrects and extends the original description(s). Then, we use a novel method to compare compatible fragments of process information to arrive at inter-process distance. This information is now used to analyze all the processes and provide logical clusters of similar processes. The method has been tested on a dataset consisting of hundreds of processes documented in Word and Visio, and

could find meaningful process clusters. We validate the usefulness of process clusters by utilizing them to find related non-process information fragments. In an empirical evaluation on a large set of process artifacts, we found similar processes also exhibit a high degree of similarity in the ancillary non-process information pieces that are linked to them. Thus, when a consultant is interested to find *performance indicators* in context of the *Accounts Receivable* process, he may find performance indicators written for other processes belonging to the same cluster as *Accounts Receivable* to be particularly useful. The benefit of our approach is that enterprises can now get a more complete picture of their processes while the users can continue to use multiple representations as their methodology demands.

The contributions of the paper are that we:

- provide a method to link a set of processes' textual and graphical (flow) descriptions together
- provide a method to enhance, correct and normalize the description of a business process by using both its flow and textual documentation
- provide a method to discover process clusters using their documentation
- empirically prove that the degree of similarity of ancillary non-process information pieces is higher within process clusters
- demonstrate that the process clusters are helpful in practice in applications like search

In the remainder, we first give some motivating examples and analyze the problem. We then discuss our solution. Next we describe performance of the solution with experiments. We conclude with a discussion of related work and future directions.

2 Motivation

Let us consider a situation where business process information is used heavily – implementation of a packaged Enterprise Resource Planning (ERP) system. A packaged ERP application provides common business functions and processes, embodies industry best practice processes, allowing an enterprise to leverage experience of others. An ERP implementation can easily touch upon hundreds of business processes. Standard process taxonomies like APQC's Process Classification Framework (PCF)[1] provide basic knowledge of all possible business processes but not enough details to implement an IT transformation like ERP. Consultants

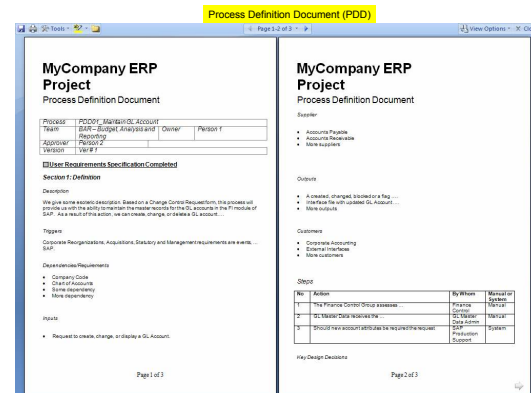


Figure 1. Input Word file of running example.

in ERP service engagements have to chart out all business processes of the client in great detail and create extensive blueprints for realizing transformations. Hence, organizing process documentation of such projects is of high importance.

Consider the example of a business process documented in a textual format on a large ERP project. The document, sometimes called Process Definition Document (PDD), covers information on process name and description, details of process steps, inputs and outputs to every process step as well as non-process information such as suppliers, performance indicators, risk considerations, compliance requirements, etc. Figure 1 shows an example PDD. Note that process steps are detailed in a table with attributes like step name, role and characteristic of activity.

The benefit of textual description is that all aspects of the process can be explained in detail without any restriction on format or page size. However, behavioral aspects of the process (i.e., the control and data flow among its steps) may be more succinctly captured in a graphical (flow) description of the process using a simple-to-use tool like Visio. In Figure 2, the flow description for the example process is shown. The benefits of flow representation is that the control flow is detailed and intuitive. However, a drawback is that the process description has to be restricted due to physical limitation of screen and pages. Also, non-process information pieces are not usually linked to flow descriptions. Nevertheless, having at least one textual description of the process and another graphical description is very common.

Having multiple representations can invariably lead to inconsistencies, especially in ERP, as many people are involved in a project and the descriptions can be from different people with differing assumptions. For example, the names in graphical and textual descrip-

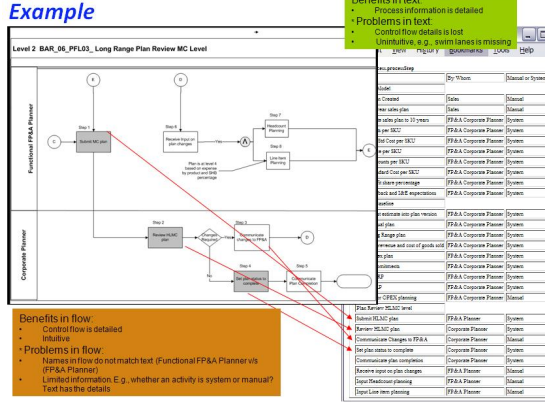


Figure 2. An example of issues and enhancements possible from textual and diagrammatic descriptions.

tions of our example process do not match (Functional FP&A Planner v/s (FP&A Planner)). Also, information in a diagram can be omitted due to space reasons. E.g., "Whether an activity is system or manual?" – the text sample has the details but the flow does not.

To summarize, enterprises contain a lot of process information implicit in documents (semi-structured or unstructured text and flow diagrams), but they are not strung together to present a comprehensive view of the state of business. Organizing related process information can provide a single view of truth in context of a particular engagement and can also help to assestize process artifacts and reuse them across projects. Again, linking ancillary non-process information to process elements and maintaining them in the same light can be beneficial in boosting reuse of non-process (but often critical) design information.

3 Problem Formulation

In a typical project, we will have a large collection of process documents (PDs), $\mathbf{D} = d_1 \dots d_n$. The documentation of a process p_i is referred to by d_i . The document consists of different information pieces tagged by labels, $\mathbf{T} = \{t_1, \dots, t_k\}$. A publicly available tool that can help tag documents created by Microsoft Word is IBM Content Harvester¹. We will denote the content of an information type t_j in document d_i by $d_i(t_j)$. Now the content $d_i(t_j)$ can be anywhere in the document, e.g., inside a section, sub-section, cell of a table.

The project can also decide to document the graphical representation of the process' behavior, called *process flow*, either inside the process document or separately. A process flow is made up of nodes representing activity, edges representing dependency, and op-

tionally additional annotations like business objects, resources, organizations, etc. If the process flow is inside the document, it has the tag *BusinessProcess.flow*. If the process flow is kept separate from process' textual description, and not within it, we have $D^f = d_1^f \dots d_n^f$ in addition to $\mathbf{D}$. We call d_i^f as process flows (PFs).

All $d_i(t_j)$, other than process flows, can be structured as any one from the following:

- a continuous text, called *single content*
- a list of text, called *content list*
- a table of text, called *content table*

Example 1: Consider Figure 1. Content for different tags are in table cells and sections of the document. The tags t_i s are *BusinessProcess.description*, *BusinessProcess.KPI*, *BusinessProcess.requirements*, *BusinessProcess.steps*, *BusinessProcess.inputs*, *BusinessProcess.outputs*, *BusinessProcess.triggers*, etc. The content $d_{example1}(\text{BusinessProcess.requirements})$ is a list in Figure 1 while $d_{example1}(\text{BusinessProcess.steps})$ is a table in Figure 2.

For a project, let $\mathbf{D}$ and D^f represent the process documents and process flows created, respectively. The problems we consider are:

- *Linking* problem: Link each $d_i \in \mathbf{D}$ with the corresponding $d_i^f \in D^f$.
- *Normalization* problem: Have the information about the process steps in a consolidated form across text $d_i(\text{BusinessProcess.steps})$ and flow, d_i^f .
- *Grouping* problem: Find groups (clusters) of similar processes based on their documented details, i.e., find $\mathbf{G} = \{g_1, \dots, g_k\}$ where $g_i \subseteq \mathbf{D}$ such that when $d_m, d_n \in g_i$, $p_m \approx p_n$.

4 Solution Approach

We now discuss how each of the problems are addressed.

4.1 Linking Process Information

The first step in organizing processes through the proposed solution involves preparing process repositories. Service vendors accumulate a large number of text documents as well as diagrams depicting business processes that they configure as part of ERP engagements. The rich business process information captured in these documents may be effectively reused in future engagements after cleansing any client specific information present in them. We scan such repositories

¹At: <http://www.alphaworks.ibm.com/tech/contentharvester>

of documentation to establish linkages between existing designs that pertain to identical or similar business processes.

A simple method of comparing a textual listing of process steps with a process flow diagram involves computing the fraction of “common” steps between them. Two process steps s_i and s_j are said to be equivalent if they have similar names and also exhibit demonstrate a high degree of aggregated similarity in names of their predecessors(P), successors(S), inputs(I) and outputs(O).

$$\theta_a(s_i, s_j) = w_t \times \theta_t(s_i, s_j) + \sum_X \sum_{s_i^n \in X(s_i)} \sum_{s_j^n \in X(s_j)} sim_t(s_i^n, s_j^n) \quad (1)$$

where, θ_t may be any text similarity measure viz. Levenshtein, Monge Elkan etc, and $X = P, S, I, O$, and θ_a is the overall similarity. Finally, the two listings match if the fraction of similar steps is significant as per user defined thresholds.

Although, the above method works well in practice one can also define more sophisticated measures of comparing process flows based on ontologies [4] and interesting graph edit distances [9].

4.2 Normalizing Steps

As motivated in Section 1, business process of the client is central to all activities in any ERP engagement. With process centric methodology gaining momentum, the business process description is getting captured in increasingly greater detail. Again, the process description often comes in various manifestations both as text and as diagrams. Thus, a canonical representation of the business process flow may yield the following benefits:

Enrichment of process information: Process step information is present in varying degree of detail across different kinds of documents. Whilst, the process flow diagram crisply depicts the dependencies between the steps, a detailed textual description of the activities performed as part of a step is available in a process definition document. We aim to consolidate these pieces of information at one place in a normalized representation of a process step.

Consistency and a single view of truth: Since, same information is contained in multiple documents, which are constantly being updated, guaranteeing the fact that the information being referred is not stale becomes a challenge. A single view of truth having consolidated information can help in alleviating inconsistencies.

More structured information: A structured representation would enable greater precision in harvesting process information for search and other purposes.

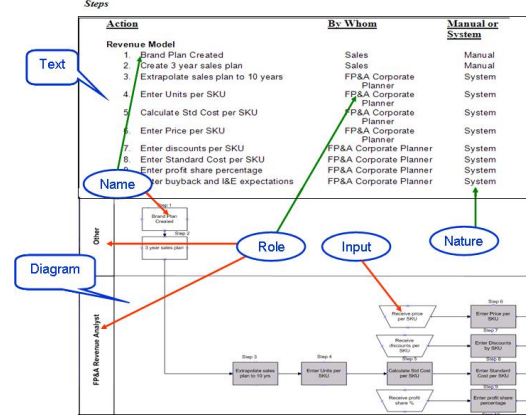


Figure 3. Features of a Process step

We propose a normal form of process step that contains placeholders for all facets of information that are usually linked to a step. The normal form contains the following:

1. *Name:* A short name for the step.
2. *Description:* A detailed description that enables all further design/realization pertaining to the step.
3. *Role:* The entity (may be person/organization/machine) responsible for realizing the step.
4. *Predecessors:* The set of steps preceding the step in question.
5. *Successors:* The set of steps following the step in question.
6. *Input:* Data input (if any) to the step.
7. *Output:* Data output (if any) from the step.
8. *Nature:* Whether the step is manual/automatic
9. *Miscellaneous:* A placeholder for any other kind of information.

Typically, a diagrammatic representation of the process flow will unambiguously capture name, role (through a swimlane), predecessors, successors, input and output for every step. The detailed step description may be mined from the text in process definition documents. Figure 3 shows table of process steps present in a Process definition document and contrasts it with the corresponding process flow diagram. We find that certain features are present only in one form of document and thus there is scope of enrichment of

process information through consolidation. For example in Figure 3, the nature of a step is captured solely in text whereas the inputs for a step are explicitly indicated only in the flow diagram. Inputs and outputs for a step may be specified as nodes in the flow graph or as labels on the incoming and outgoing edges of a step node. Also, notice that the role responsible for Steps 1 & 2 is “Sales” according to the step table whereas this role is under-specified (stated to be “Others”) in the process flow. In such cases where merging of the same feature becomes non-trivial we can seek human intervention. Using advanced text analytics to automatically discover complex merging rules can be an interesting research problem.

4.3 Grouping Processes

For grouping processes based on their documentation, clustering algorithms [5] provide a natural solution framework except that the distance function has to be provided between (linked and normalized) processes documentation. We detail our selected clustering algorithm and our novel method to measure distance between processes. The main steps are shown in Figure 4. The steps consist of first finding pairs of similar processes using the documents’ comparably tagged contents and then using pair-wise similarity with standard transitive closure techniques to build clusters of overall equivalent processes. Since content for a tag itself can be structured in different ways, the exact matching steps need to take care of content structuring (Sec. 4.3.1). The scores are then aggregated to determine complete pair-wise document similarity (Sec. 4.3.2).

We note that there are many alternatives for building similarity scores and the selection of one approach over another only reflects in the quality of the overall clusters. Our aim is to demonstrate the overall feasibility and usefulness of process comparison approach; finetuning to pick the best measure for an application is a topic of future research. The rest of the section gives detail of how the similarity of documents is determined from their tags and content structure.

4.3.1 Comparing Content of Same Tag

Comparing Two Single Contents

Let $\theta(s_i, s_j)$ represent the similarity score of two single contents (strings), s_i and s_j . There are many choices for θ . Let $\supseteq$ be the binary *substring* operator to check if the first argument contains the second argument. We define one measure

$$\theta_1(s_i, s_j) = \begin{cases} 1, & \text{if } (s_i \supseteq s_j) \cup (s_j \supseteq s_i) \\ 0, & \text{otherwise} \end{cases} \quad (2)$$

```

Algorithm: FindSimilarProcessesFromDocs
Inputs: $D: a set of process documents with their
content pieces tagged,
 $\varphi$ : a threshold for document similarity
Output: Set  $G = \{g_1, \dots, g_k\}$  where
 $g_i \subseteq D$ 

Main Steps
1. Let  $\$SP = \text{FindSimilarProcessPairsFromDoc}(\$D, \varphi)$ 
2.  $G = \text{BuildClustersbyTransitiveClosure}(\$SP)$ 

Sub-Routine: FindSimilarProcessPairsFromDoc
Inputs: $D: a set of process documents with their
content pieces tagged
 $\varphi$ : threshold
Output: Set  $\$SP$  of similar document(process) pairs
1. For each pair of documents
2. For each tag in both documents
3. Compare content and record their similarity score  $\theta$  (Sec. 4.3.1)
4. Aggregate similarity score for all tags
5. Compare score with threshold and decide similarity (Sec. 4.3.1)
6. Build list of pair of similar process documents and return

Sub-Routine: BuildClustersbyTransitiveClosure
Inputs:  $\$SP$ : a set of similar process pairs
Output: Set  $G$ 
1.  $\$Clusters = \{\}$ 
2. For each pair in passed  $\$SP$ 
3.  $\$seed = \text{pick first process of pair}$ 
4.  $\$ACluster = \text{BuildAClusterWithGivenSeed}(i, \$SP)$ 
   ;; Note:  $\$SP$  can get reduced by side-effect.
5. Add  $\$ACluster$  to  $\$Clusters$ 
6. Return  $\$Clusters$ 

Sub-Routine: BuildAClusterWithGivenSeed
Inputs:  $\$seed$ : a process all of whose neighbors have to be found
 $\$SP$ : a set of similar process pairs
Output: Set  $g_i$ 
1.  $\$ClusterOfSeedProcess = \{\}$ 
2. For each process  $i$  similar to seed process
3. Add  $i$  to  $\$ClusterOfSeedProcess$ 
4. Remove pair from  $\$SP$ 
5. For each process  $j$  in  $\$ClusterOfSeedProcess$ 
6.  $\$NewMembers = \text{BuildAClusterWithGivenSeed}(j, \$SP)$ 
7. Add  $\$NewMembers$  to  $\$ClusterOfSeedProcess$ 
8. Return  $\$ClusterOfSeedProcess$ 

```

Figure 4. Pseudo-code of algorithms to group processes.

We also use another measure called Monge-Elkan (θ_2) from the many standard string matching metrics available in [3]. θ_1 is more strict than θ_2 but faster to compute.

Now s_i is similar to s_j , denoted by $s_i \approx s_j$, if $\theta(s_i, s_j) > \phi$, where ϕ is some threshold.

Comparing Single Content and Content List

Let $\theta(s_i, l_j)$ represent the similarity score of a single content s_i with a content list l_j . We use the measure:

$$\theta_{sl}(s_i, l_j) = \begin{cases} 1, & \text{if } (s_i \approx s_k^l | \exists s_k^l \in l_j) \\ 0, & \text{otherwise} \end{cases} \quad (3)$$

Now s_i is similar to l_j , denoted by $s_i \approx l_j$, if $\theta_{sl}(s_i, l_j) \neq 0$.

Comparing Single Content and Content Table

Let $\theta(s_i, t_j)$ represent the similarity of a single content s_i with a content table t_j . We use the measure:

$$\theta_{st}(s_i, t_j) = \begin{cases} 1, & \text{if } (s_i \approx s_k^l | \exists s_k^l \in t_j) \\ 0, & \text{otherwise} \end{cases} \quad (4)$$

Now s_i is similar to t_j , denoted by $s_i \approx t_j$, if $\theta_{st}(s_i, t_j) \neq 0$.

Comparing Two Content List

Let $\theta(l_i, l_j)$ represent the similarity score of two content lists, l_i and l_j . We use the measure:

$$\theta_{ll}(l_i, l_j) = \frac{|(s_i \approx s_j) | s_i \in l_i, s_j \in l_j|}{\text{Max}(|l_i|, |l_j|)} \quad (5)$$

Now l_i is similar to l_j , denoted by $l_i \approx l_j$, if $\theta_{ll}(l_i, l_j) \neq 0$.

Comparing Content List and Content Table

Let $\theta(l_i, t_j)$ represent the similarity score between the content list l_i and content table t_j . We use the measure:

$$\theta_{lt}(l_i, t_j) = \text{Max}(\theta(l_i, t_t^k) | t_t^k \in t_j) \quad (6)$$

That is, we consider it to be the maximum score between the list and any column of the table. Now l_i is similar to t_j , denoted by $l_i \approx t_j$, if $\theta_{lt}(l_i, t_j) \neq 0$.

Comparing Two Content Tables

Let $\theta(t_i, t_j)$ represent the similarity score of two content tables, t_i and t_j . We use the measure:

$$\theta_{tt}(t_i, t_j) = \begin{cases} \text{If}(t_i^{\text{cols}} == t_j^{\text{cols}}), \\ \text{Max}(\theta(s_i, s_j) | s_i \in t_i, s_j \in t_j) \\ 0, \text{otherwise} \end{cases} \quad (7)$$

Now t_i is similar to t_j , denoted by $t_i \approx t_j$, if $\theta_{tt}(t_i, t_j) \neq 0$.

4.3.2 Comparing Two Process Documents

The similarity of two processes documents, d_i and d_j , is measured by

$$\theta_{dd}(d_i, d_j) = \frac{\sum_{a=1..k} |d_i(t_a) \approx d_j(t_a)|}{\text{Max}(|d_i(\mathbf{T}_i)|, |d_j(\mathbf{T}_j)|)} \quad (8)$$

That is, the similarity score is computed by aggregating the similarity score of content of common tags in both documents. d_i is similar to d_j , denoted by $d_i \approx d_j$, if $\theta_{dd}(d_i, d_j) > \varphi$. If $d_i \approx d_j$, then p_i is similar to p_j .

5 Experiment

We carried out experiments on a set of 240 process definition documents and 315 process flow diagrams that were authored in the context of a large ERP engagement. It was not known if all process documents had the corresponding process flows or vice-versa. The aim of our experiments was to check the effectiveness of our methods for the *link*, *normalize* and *group* problems, and show the benefit of the results.

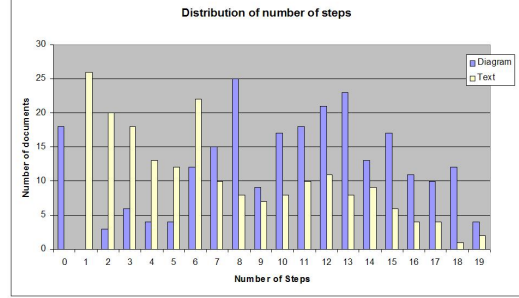


Figure 5. Distribution of number of steps in process documents

5.1 Linking process documents and flows

The first step in our empirical evaluation involved trying to establish link across process documents (PD) and process flow (PF). However, first we had to see the characteristics of the two types of documents to set the right result expectation. Figure 5 show a distribution of the number of steps in the diagrammatic and textual descriptions of business processes.

As we can see,

- The step description in PD and PF follow different distribution of steps. The PDs tend to have small number of steps while PFs have large number of steps.
- We expect PDs to be without corresponding PFs and vice-versa.

From the data, we considered only those processes (171 PDs and 294 PFs) that have greater than 2 step for this purpose. We experimented with a host of standard text similarity measures to match step names across processes captured in these two sets. We have used the Secondstring [3] library of similarity measures available in Java. The Jaro similarity measure which is known to be effective in matching short string records gave us best results. We carefully set the threshold for defining a match in step names so that we have a fair degree of precision and recall. We also use the exact string comparison function in Java. Table 1 has the results. Here, candidate matches refers to the number of PD-PF matches that were above minimum threshold to be considered candidates for links. We find a total of 126 matches involving 30 PDs. We have manually verified the validity of the matches in order to compute precision to be 48%. When we use exact string comparison to match step names, the method found matches for only 11 PDs (all of them match 1 flow each) but we get 100% precision. Since we do not know how many PDs had PFs in the collection for sure, recall cannot be calculated.

String Measure(θ)	# Candidate Matches	PDs	Precision (%)
θ_{Jaro}	126	30	48
θ_{Exact}	11	11	100

Table 1. Results in PD-PF link experiment

String	% Match (Name)	% Match (Name+Role)
θ_{Jaro}	37	8
θ_{Exact}	45.5	13

Table 2. Results in automatic PD-PF normalization experiment for matching documents.

We note from the results that accuracy of the method depends on the string measure but can be high. However, since the candidate number of matches were small, the recall is likely to be low. This indicates that the method can be a helpful guide to someone who is manually linking PDs to PFs but in itself, the method will not be able to link a lot of data. Further evaluation is needed with other datasets.

5.2 Normalizing process step information

We now conduct experiment to check if there is value in automatically trying to normalize the process information in the document and flow for those which matched as output of the linking method. In the dataset, there were 3 features in PD and 6 features in PF while the normalized output has 7 features. Step *name* and *role* are common attributes across PD and PF while *description* and *miscellaneous* are not available in the dataset after *linking* experiment. We already note than any step that we can match and normalize will have more information than what would have been seen in any individual document.

Table 2 has the results. The results show that there is a good potential for automatically normalizing steps across PDs and PFs.

5.3 Finding Clusters of Processes

We looked at 240 process documents from an engagement. They were harvested with Content Harvester and different information types were tagged. The threshold for document similarity φ was set at 20% match level while θ_1 was used for string similarity. The output is shown in Figure 6.

Here are some statistics about the output:

- Number of pair-wise similar processes = 266
- Number of clusters found = 23

- Range of cluster sizes = (2, 21)

- Number of documents in some cluster = 134 (i.e., 55% of total)

We note that only slightly more than half of the processes are similar to some other process, but there are large groups of similar processes. The groupings were manually inspected for correctness using other information from the original project, and they were mostly correct².

5.4 Benefit of Discovered Clusters

Now we show that the process grouping information can be used for beneficial applications. Among all the information types that is documented on processes for a project, there are many which are process-independent. Consultants frequently want to answer queries like, “find all requirements across all available processes that are similar to those of the Account Receivable process”. We now report for different information types (tags), the average similarity of non-process content between processes in the same cluster versus any random two process.

Table 3 shows the results. Column 2 has $Avg(\theta(d_i(t_k), (d_j(t_k))), d_i, d_j \in \exists g_m)$ while Column 3 has $Avg(\theta(d_i(t_k), (d_j(t_k))))$. Column 4 is (Column2-Column3/Column3) as percentage. We see that for all tags we evaluated, the average similarity within the cluster was 2-3 orders of magnitude higher than what was present without our approach. Hence, any search application for processes can boost its performance by using just the process clustering information before any input is sought from the user. Since it is intuitively true that same/similar non-process design information shall be linked to similar processes, the experiment also proves that the clusters found by our approach indeed group meaningful processes together.

6 Discussion and Related Work

APQC’s PCF[1] (or just APQC as it is popularly called) is a standard for terminology on process definitions and measures for benchmarking. APQC can be used in many contexts where people, data and IT need to be organized around the business area that they impact. Similarly, software vendors like SAP also provide high-level process documentation via their tools like SAP Solution Composer. However, such high-level documentation states what a business can broadly do

²Measuring absolute correctness is not possible since the original experts of the project are unavailable.

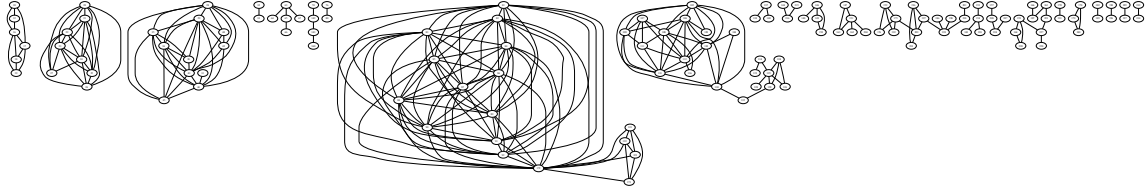


Figure 6. Clusters discovered among 240 process documents from a live project. Each node represents a document.

Information Sub-Type (Tag)	Avg. in Cluster	Avg. Overall	Similarity Boost (in %)
BusinessProcess.requirement	0.209	0.014	1430.55
BusinessProcess.integrationConsideration	0.620	0.115	438.54
BusinessProcess.leanSigma.input	0.242	0.0336	620.75
BusinessProcess.leanSigma.output	0.228	0.011	1941.50
BusinessProcess.leanSigma.supplier	0.844	0.109	671.22

Table 3. Effectiveness of discovered clusters in boosting similarity of process’ information sub-types (tags).

whereas process documents as we considered are a detailed account of how a business function will be implemented and performed.

There is extensive literature on similarity of business process models [4, 2] and equivalence of process structures [6]. However, they only work with well defined process workflow structures and do not consider process documentation available as semi-structured or unstructured text. Again, process mining efforts [8] and tools [7] have focused on building processes from execution traces. Unlike our work, prior art in BPM does not try to mine process information from documents.

In the presented work, we did not assume any metadata from the repository that stored the process documentation. While this harder situation is a reality in our setting, if such information were available, it would simplify the problem. Specifically, it will (a) reduce or even eliminate *linking* sub-problem where we associate corresponding text and flow documentation, and (b) could be used while solving the *grouping* sub-problem since processes known to originate from same repository category can be given more weight while comparing similarity.

7 Conclusion

In this paper, we considered the problem of organizing process information that exist in enterprise repositories. Our solution consists of first reconciling process description from multiple representations to produce an enhanced process definition that corrects and extends the original description(s). Then, we define a novel method of computing inter-process distance using compatible fragments of process information and build meaningful clusters of similar processes. We tested our solution on a large dataset consisting of hundreds

of processes documented in Word and Visio, where it could link processes’ textual and flow documents, normalize step information and find process clusters that significantly boosted search performance. Our future work will involve experimenting with more string matching alternatives and datasets.

References

- [1] APQC. Apqc cross industry process classification framework (pcf), ver.5.0.3. In *www.apqc.org.*, 2008.
- [2] A. Awad, A. Polyvyanyy, and M. Weske. Semantic Querying of Business Process Models. In *Proc. 12th Intl IEEE Enterprise Distributed Object Computing Conf.*, pages 85–94. Washington, DC, USA, 2008.
- [3] W. Cohen and P. Ravikumar. Secondstring: An open-source Java toolkit of approximate string-matching techniques. Project web page, 2003.
- [4] M. Ehrig, A. Koschmider, and A. Oberweis. Measuring similarity between semantic business process models. In *Proc. 4th Asia-Pacific Conf. on Conceptual modelling*, pages 71–80. Darlinghurst, Australia, 2007.
- [5] A. K. Jain and R. C. Dubes. Algorithms for clustering data. In *Prentice Hall Publ.*, 1998.
- [6] W. van der Aalst, A. de Medeiros, and A. Weijters. Process equivalence: Comparing two process models based on observed behavior. *Lecture Notes in Computer Science*, 4102:129, 2006.
- [7] B. van Dongen, A. de Medeiros, H. Verbeek, A. Weijters, and W. van der Aalst. The ProM framework: A new era in process mining tool support. *Application and Theory of Petri Nets*, 3536:444–454, 2005.
- [8] A. Weijters and W. Van der Aalst. Process mining: discovering workflow models from event-based data. In *Proc. 13th Belgium-Netherlands Conf. on AI (BNAIC 2001)*, pages 283–290, 2001.
- [9] K. Zhang. A constrained edit distance between unordered labeled trees. *Algorithmica*, 15(3):205–222, 1996.